

Week 1 — Day 3: Daily Exercise

Signal Intercept Log

Standalone exercise. Do not copy today's code examples — use only their ideas. Before writing any code: write a plan on paper — which classification values go in the closed set, where the signal strength may be unknown, and which function updates an existing value through `ref`. Open the editor only after the plan is written.

Intelligence Need

A listening station picks up short-wave transmissions from different sources all through the shift. Each transmission has a source id, a classification (is the source friendly, hostile, or still unidentified), and sometimes a received signal strength in megahertz.

The problem: not every transmission gives a strength reading. Sometimes the signal is too weak, or the receiver has not locked yet, and then the strength is simply **unknown**. This is very different from a strength of zero, which means total silence. "Not measured" and "measured as zero" are two different things, and you must never mix them up.

The classification must be chosen from a closed, fixed set — a typo must not turn a `Hostile` source into `Hostle` without anyone noticing. The classification values are shown in English in the program output: `Friendly / Hostile / Unidentified`.

One more thing: the first reading is usually rough. After the receiver settles, the operator runs a **calibration** on an existing reading — it takes the current strength value and sharpens it in place, on the same variable.

Functional Requirements

- Represent the source classification with a closed, fixed set of values — no free strings in the code. The classification values are shown in English in the output: `Friendly / Hostile / Unidentified`.
- A signal strength that may be unknown must be represented so you can tell "no reading" apart from "strength zero". In the display, show a clear value (such as `unknown`) when there is no reading.

- Provide a **calibration** action that takes an existing strength variable through `ref` and updates it in place (for example: sharpens or corrects the existing value). Note — the action changes the caller's own variable, not a copy of it.
 - Let the operator: log a new transmission, calibrate the strength of an existing source, and list all sources with their classification and strength (showing `unknown` where the strength was not measured).
 - Offer the operator a menu that repeats until they choose to quit.
-

Direction Hints (not a solution)

- Think about the difference between a variable passed to a function as a copy and a variable passed so that the change comes back to the caller — what is the difference in behavior, and why does calibration need the second one?
 - Think about how "no reading" is technically different from "zero", and which tool in the language lets you hold both states on the same numeric variable.
 - Think about why a closed set of classification values is better than a free string — what gets caught at compile time, and what would have slipped through silently with a string.
-

Self-Assessment

Skill	Status
I represented the classification with a closed set of values, no free strings	
I represented a strength that may be unknown and told it apart from zero	
I wrote a calibration action that updates an existing variable through <code>ref</code>	
I can explain why calibration needs a pass that returns the change to the caller	
I built an operator menu for logging, calibrating, and listing	